DSA711S Data Structures and Algorithms 2

Binary Search Tree and AVL Tree

2026

Overview

- A binary tree possesses ordering property that maintains the data in its nodes in sorted order.
  - Since the search tree is a linked structure, entries may be inserted and deleted without having to move other entries over, unlike ordered lists in which insertions and deletions require data movement.
  - The AVL tree is a height-balanced binary search tree that delivers guaranteed worst-case search, insert, and delete times that are all O(log n)

Learning Objectives

- Explore the motivation for binary search trees by learning about the comparison tree for binary search.
- Use the comparison tree as an analytical tool to determine the running time of binary search.
- Describe the binary search tree structure and properties.

- Study the primary binary search tree operations of search, insert, and delete, and analyze their running times.

Learning Objectives

- Understand a binary search tree class interface and use it in application examples.
- Implement the binary search tree class with a binary tree class as the reused storage component.
- Study the AVL tree structure properties, the search, insert, and delete operations, and their running times.

10.2 Binary Search Tree Properties

What we expect from a binary search tree

Insertions and deletions should be faster than $O(n)$, and search time should not be slower than $O(\log n)$.

Binary search tree

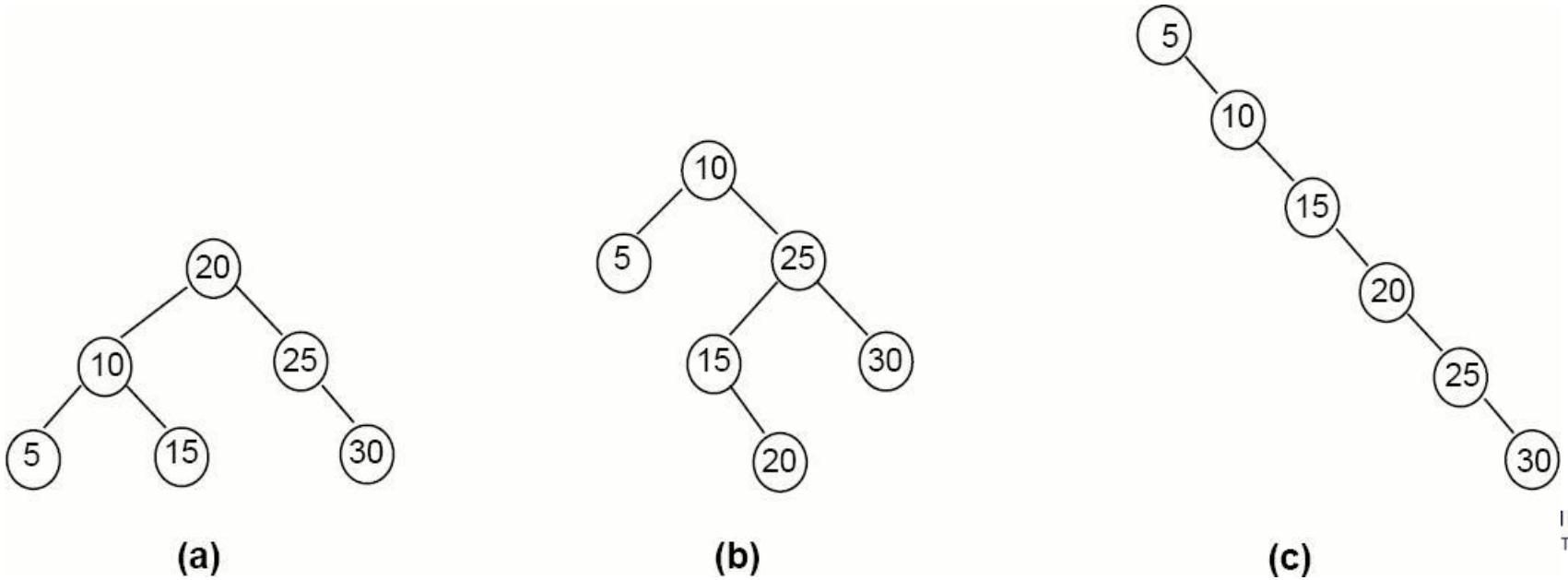
A binary search tree is a binary tree whose entries can be arranged in order. For every node x in the tree, the value of the entry at x is greater than the values of all the entries in the left subtree of x, and smaller than the values of all the entries in the right subtree of x.

Inorder Traversal of a binary search tree

The inorder traversal of a binary search tree visits the nodes in ascending order of values.

10.2 Binary Search Tree Properties

- All three trees have the same set of keys.
- Their structures are different, depending on the sequence of insertion or deletion.



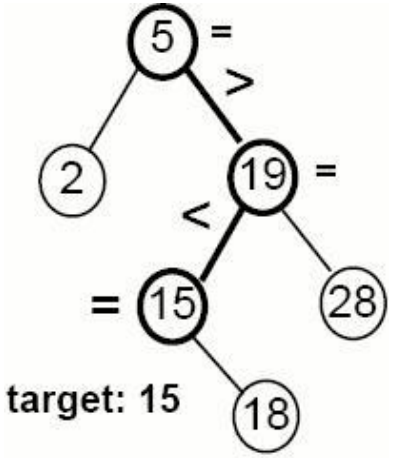
10.3 Binary Search Tree Operations

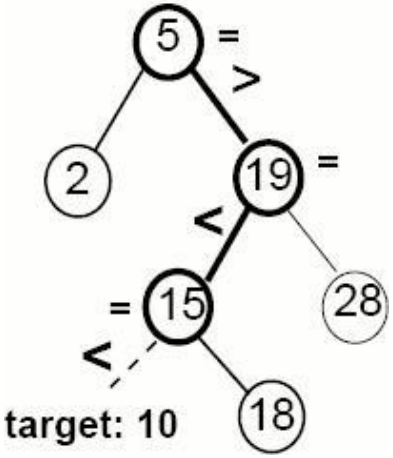
- Three foundational operations
  - Search
  - Insert
  - Delete

10.3.1 Search

- The tree nodes are for real.
  - The target key is compared against the key at the root of the tree.
  - If they are equal, sucess.
  - If not, recusively search the appropriate child.
  - Search terminates with failure if an empty subtree is reached.

10.3.1 Search

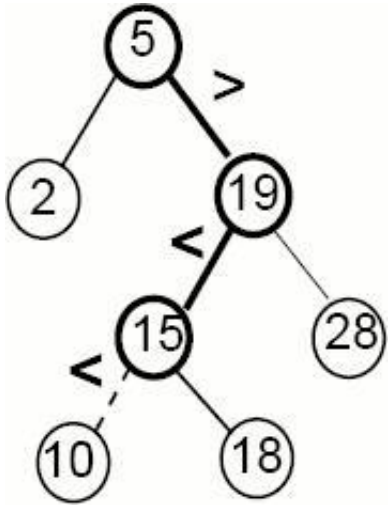




10.3.2 Insert

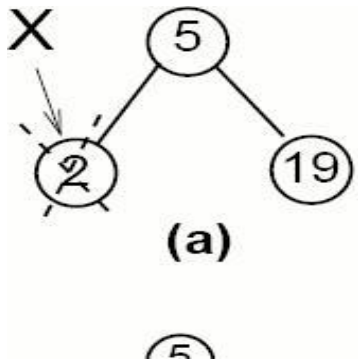
- To insert a value, search must force a failure.
- Item in inserted in the failed location.
- A newly inserted node always becomes a leaf node in the search tree.

10.3.2 Insert



10.3.3 Delete

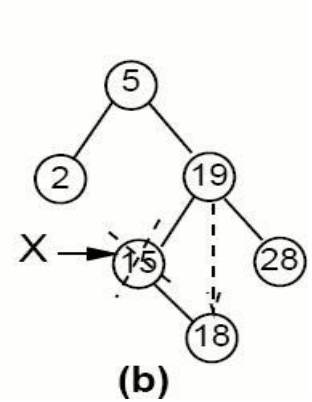
- The value to be deleted is first located in the binary search tree.
- Three possible cases.
  - Case a: *X* is a leaf node.



(a)

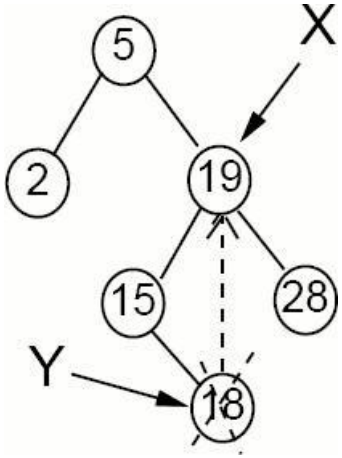
10.3.3 Delete

- Case b: *X* has one child
  - Replace the deleted node with the child.



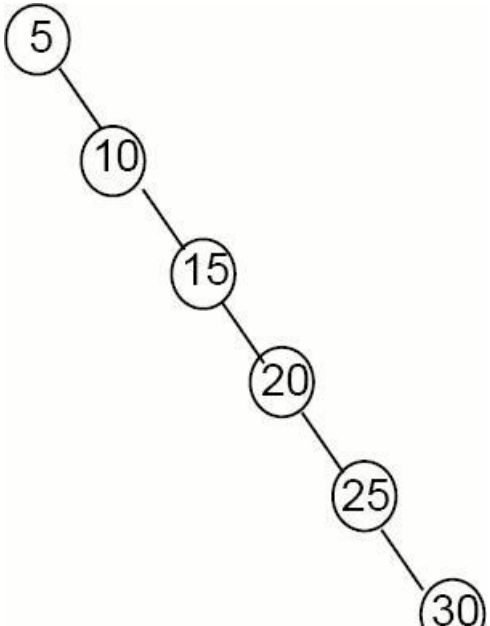
10.3.3 Delete

- Case c: *X* has two children
  - Find the inorder predecessor, *Y*, of *X*.
  - Copy the entry at *Y* into *X*.
  - Apply deletion on *Y*.
    - Applying deletion on *Y* will revert to either case *b* or *a* since *Y* is guaranteed to not have a right subtree.



Running Times

- Search: worst case
  - Tied to the worst possible shape a tree can attain.
  - Such a tree degenerates into sequential search.
    - O(n).



Running Times

Skewed tree

The worst possible binary tree structure is one that is completely *skewed* either to the left (no node has a right child), or to the right (no node has a left child). This happens when the values to be inserted arrive in descending or ascending order, respectively. Searching in such a tree degenerates to sequential search, with the attendant time of $O(n)$.

Running Times

- Insertion: worst case
  - O(n)
- Deletion: worst case
  - O(n)

Operation	Unordered List	Ordered List	Binary Search Tree	
Search	$O(1)$	$O(1)$	$O(1)$	Best
	$O(n)$	$O(\log n)$	$O(n)$	Worst
Insert	$O(1)$	$O(\log n)$	$O(\log n)$	Best
	$O(1)$	$O(n)$	$O(n)$	Worst
Delete	$O(1)$	$O(\log n)$	$O(\log n)$	Best
	$O(n)$	$O(n)$	$O(n)$	Worst

Balancing

- Keeping a binary search tree balanced allows the height never to exceed O(log n).
- There are two popular ways of maintaining and constructing balanced binary search trees.
  - AVL tree.
  - red-black tree.

10.4 A *BinarySearchTree* Class

Class structures.tree.BinarySearchTree<T extends Comparable<T>>

Constructors

BinarySearchTree( )
Creates an empty binary search tree.

Methods

$O(n)$ **void insert(T item)**
Inserts the specified item into this binary search tree.
Throws **OrderViolationException** if this is a duplicate key insertion.

$O(n)$ **void delete(T key)**
Deletes the item from this tree with the given key, and returns the deleted item.
Throws **NoSuchElementException** if no item has given key.

$O(n)$ **T search(T key)**
Returns the item in this tree with the given key, null if no item has this key.

$O(1)$ **isEmpty( )**
Returns true if tree is empty, false otherwise.

10.4 A *BinarySearchTree* Class

$O(1)$ **int size( )**
Returns the number of items in this tree.

$O(n)$ **T minKey( )**
Returns the minimum-key item in this tree.
Throws **NoSuchElementException** if tree is empty.

$O(n)$ **T maxKey()**
Returns the maximum-key item in this tree.
Throws **NoSuchElementException** if tree is empty.

$O(n)$ **void preOrder(Visitor<T> visitor)**
Visits the tree in preorder.

$O(n)$ **void inOrder(Visitor<T> visitor)**
Visits the tree in inorder.

$O(n)$ **void postOrder(Visitor<T> visitor)**
Visits the tree in postorder.

10.5.1 Example *Treesort*

```
BinarySearchTree<Integer> B = new BinarySearchTree<Integer>();
B.insert(15);
B.insert(8);
B.insert(10);
Visitor<Integer> visitor = new Visitor<Integer>();
B.inOrder(visitor);    // prints 8, 10, 15 in that order
```

- *inOrder* traversal method invokes *visitor.visit()* when a node is visited.

10.5.1 Example: *Treesort*

Treesort algorithm

Insert the entries into a binary search tree, one by one, and when they have all been inserted, perform an inorder traversal on the tree, to obtain the entries in sorted order.

10.5.2 Example: Counting Keys

- Count the number of keys in a binary search tree that are less than a given key.
- It is possible to simply examine every node of the tree.

10.5.2 Example: Counting Keys

```
public class CountVisitor<T extends Comparable<T>> extends Visitor<T> {

    private int count;
    private T key;

    public CountVisitor(T key) {
        this.key = key;
        count = 0;
    }

    // overrides the superclass visit method
    public void visit(BinaryTree<T> node) {
        if (node.getData().compareTo(key) < 0) count++;
    }

    public int getCount() {
        return count;
    }
}
```

10.5.2 Example: Counting Keys

- Counting integer values that are less than 50.

```
BinarySearchTree<Integer> B = new BinarySearchTree<Integer>();
// insert a bunch of integers into B
...
CountVisitor<Integer> cv = new CountVisitor<Integer>(50);
B.preOrder(cv);
System.out.println("count = " + cv.getCount());
```

10.6 *BinarySearchTree* Class Implementation

Class File 10.1 *BinarySearchTree.java*

```
package structures.tree;

import structures.linear.OrderViolationException;
import java.util.NoSuchElementException;

public class BinarySearchTree<T extends Comparable<T>> {
    BinaryTree<T> tree;
    int size;

    public BinarySearchTree() { tree = new BinaryTree<T>(); size = 0; }
    public boolean isEmpty() { return tree.isEmpty(); }
    public int size() { return size; }
    BinaryTree<T> recursiveSearch(BinaryTree<T> root, T key) { ... }
    public T search(T key) { ... }
    public void insert(T item) { ... }
    void deleteHere(BinaryTree<T> deleteNode, BinaryTree<T> attach) { ... }
    BinaryTree<T> findPredecessor(BinaryTree<T> node) { ... }
```

10.6 *BinarySearchTree* Class Implementation

```
    public T delete(T key) { ... }
    public T minKey() { ... }
    public T maxKey() { ... }
    void recursivePreOrder(BinaryTree<T> root, Visitor<T> visitor) { ... }
    public void preOrder(Visitor<T> visitor) { ... }
    void recursiveInOrder(BinaryTree<T> root, Visitor<T> visitor) { ... }
    public void inOrder(Visitor<T> visitor) { ... }
    void recursivePostOrder(BinaryTree<T> root, Visitor<T> visitor) { ... }
    public void postOrder(Visitor<T> visitor) { ldots }
}
```

10.6.1 Search Implementation

```
public T search(T key) {
   if (tree.isEmpty()) {
      return null;
   }
   return
   recursiveSearch(
            tree, key).data;
}
BinaryTree<T> recursiveSearch(
   BinaryTree<T> root, T key) {
   if (root == null) {
      return null;
   }
   int c =
      key.compareTo(root.data);
   if (c == 0) { return root; }
   if (c < 0) {
      return
         recursiveSearch(
            root.left, key);
   } else {    // right subtree
      return
         recursiveSearch(
            root.right, key);
   }
}
```

10.6.2 Insert Implementation

```
public void insert(T item) {
    if (tree.isEmpty()) {
        tree.makeRoot(data);
        size++;
        return;
    }
    // iterative descent
    BinaryTree<T> root = tree;
    boolean done=false;
    BinaryTree<T> newNode = null;
    while (!done) {
        int c = item.compareTo(
                root.data);
        if (c == 0) {   // duplicate
            throw new
            OrderViolationException();
        }
        ...
```

10.6.2 Insert Implementation

```
while (!done) {
    ...
    if (c < 0) {
        if (root.left == null) {
        // insert as left child
            newNode =
                new BinaryTree<T>();
            root.left = newNode;
            done=true;
        } else {    // go left
            root = root.left;
        }
```

10.6.2 Insert Implementation

```
        } else {
            if (root.right == null) {
            // insert as right child
                newNode =
                    new BinaryTree<T>();
                root.right = newNode;
                done=true;
            } else {    // go right
                root = root.right;
            }
        }
    }
    // set fields of new node
    newNode.data = item;
    newNode.parent = root;
    size++;
} // insert
```

10.6.2 Insert Implementation

- *left*, *right*, and *parent* fields are directly accessed, instead of calling *attachLeft* and *attachRight*.
- Clients of *BinaryTree* that are not in its package are required to use *attachLeft* or *attachRight*.

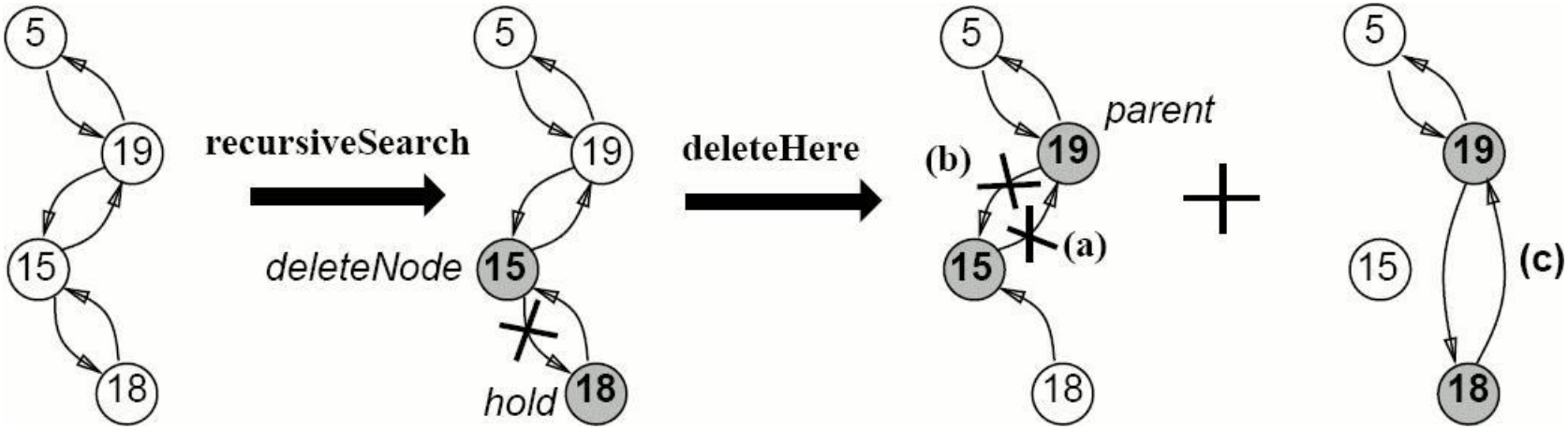
10.6.3 Delete Implementation

- *findPredecessor* helper method.
  - Case *c* requires finding the inorder predeccessor.

```
BinaryTree<T> findPredecessor(
      BinaryTree<T> node) {
   if (node.left == null) {
      return null;
   }
   // turn left once
   BinaryTree<T> pred = node.left;
   while (pred.right != null) {
      // keep going right
      pred = pred.right;
   }
   return pred;
}
```

10.6.3 Delete Implementation

- *deleteHere* helper method.



10.6.3 Delete Implementation

```
void deleteHere(
        BinaryTree<T> deleteNode,
        BinaryTree<T> attach) {
    // deleteNode has only one
    // subtree, attach
    BinaryTree<T> parent =
            deleteNode.parent;
    deleteNode.clear();
    if (parent == null) {
        return;
    }
    if (deleteNode == parent.left) {
        // attach as left subtree
        parent.detachLeft();
        parent.attachLeft(attach);
        return;
    }
    // attach as right subtree
    parent.detachRight();
    parent.attachRight(attach);
```

10.6.3 Delete Implementation

```
public T delete(T key) {
   if (tree.isEmpty()) {
      throw new
      NoSuchElementException();
   }
   // find node containing key
   BinaryTree<T> deleteNode =
      recursiveSearch(tree, key);
   if (deleteNode == null) {
      throw new
      NoSuchElementException();
   }

   BinaryTree<T> hold;

   // case c
   if (deleteNode.right != null &&
      deleteNode.left != null) {
      hold =
      findPredecessor(deleteNode);
      deleteNode.data = hold.data;
      // fall through to case 2
      deleteNode = hold;
   }
```

10.6.3 Delete Implementation

```
    // case a
    if (deleteNode.left == null &&
       deleteNode.right == null) {
       deleteHere(deleteNode, null);
       size--;
       return deleteNode.data;
    }

    // case b
    if (deleteNode.right != null) {
       hold = deleteNode.right;
       deleteNode.right = null;
    } else {
       hold = deleteNode.left;
       deleteNode.left = null;
    }

    deleteHere(deleteNode,hold);
    if (tree == deleteNode) {
       // root deleted
       tree = hold;
    }
    size--; return deleteNode.data;
}
```

10.6.4 Convenience Methods and Traversals

- *minValue* needs to find the "leftmost" node.
- *maxValue* finds the "rightmost".

```
public T minKey() {
   if (tree.data == null) {
      throw new
      NoSuchElementException();
   }

   BinaryTree<T> root = tree;
   T min=root.data;
   root = root.left;
   while (root != null) {
      min = root.data;
      root = root.left;
   }
   return min;
}
```

```
public T maxKey() {
   if (tree.data() == null) {
      throw new
      NoSuchElementException();
   }

   BinaryTree<T> root=tree;
   T max=root.data
   root = root.right;
   while (root != null) {
      max = root.data();
      root = root.right;
   }
   return max;
}
```

10.6.4 Convenience Methods and Traversals

```
public void preOrder(
      Visitor<T> visitor) {
   if (tree.isEmpty()) {
      return;
   }
   recursivePreOrder(
      tree, visitor);
}
```

```
void recursivePreOrder(
      BinaryTree<T> root,
      Visitor<T> visitor) {
   if (root != null) {
      visitor.visit(root);
      recursivePreOrder(root.left,
         visitor);
      recursivePreOrder(root.right,
         visitor);
   }
}
```

10.6.4 Convenience Methods and Traversals

- This implementation hides the tree structure from its clients.
- All clients need to see are the search, insert, and delete operation.
- "preorder", "inorder", and "postorder" provide a small window into the implementation structure.

10.9 Summary

- A comparison tree for binary search on an array is a binary tree that depicts all possible search paths.
- A failure node in a comparison tree catches a range of values that lie between its inorder predecessor and its in order successor in the tree.
- A comparison tree is simply a conceptual tool to analyze the time taken by binary search, and is therefore often referred to as an implicit search tree.

10.9 Summary

- The shape of the comparison tree is independent of the data entries in the array; it only depends on the length of the array.
- The worst-case number of comparisons for a successful search in *2h-1*, and for unsuccessful search is *2h*.
- Binary search on an ordered array is O(log n).

- A binary search tree is a binary tree whose entries are arranged in order.

10.9 Summary

- An inorder traversal of a binary search tree will visit the nodes in ascending order of values.
- The values stored in a binary search tree must lend themselves to being arranged in order.
- For any given number of values, *n*, there is only one binary search comparison tree. There are many binary search trees possible as there are different binary trees that can be constructed out of n nodes.

10.9 Summary

- The worst possible binary tree structure is one that is completely skewed either to the left or right – O(n).
- The worst-case running times for search, insert, and delete in a balanced binary search tree are all O(log n).
- Treesort is an algorithm to sort a set of values by inserting them one by one into a binary search tree, and then visiting them in inorder sequence -- $O(n^2)$.

10.9 Summary

- The AVL tree and red-black tree are two of the many types of balanced binary search trees that guarantee a worst case search / insert / delete time of O(log n).
- An AVL tree is a binary search tree in which the heights of the left and right subtrees of every node differ by at most 1.
- Recursive definition: An AVL tree is a binary search tree in which the left and right subtrees of the root are AVL trees whose heights differ by at most 1.

10.9 Summary

- Rotation about a link in an AVL tree takes O(1) time.
- Insertion in an AVL tree starts with a regular binary search tree insertion, followed by rebalancing.
- Deletion in an AVL tree starts with a regular binary search tree deletion, followed by rebalancing.